

18.3 串及KMP匹配算法

1. 串的定义

串是由零个或者多个字符组成的有限序列。串中字符的个数称为串的长度，含有零个元素的串叫空串。在C语言中，可以用如下语句定义一个名为str的串。

```
char str[] = "abcdef";
```

C语言定义了'\0'作为字符串的结束标志，但在描述串的长度时，需要通过扫描整个串才能获得，时间复杂度为 $O(N)$ ，不如额外定义一个变量专门来存储串的长度，这样求串的长度的时间复杂度为 $O(1)$ 的操作了。

不同的编程语言，是否用'\0'作为串的结束标志，是没有定论的，可以通过length来约束空间的长度也会更通用。

1.1 定长顺序存储和变长分配存储表示

串的表示

C | 复制代码

```
1 typedef struct {
2     char str[maxSize + 1]; // 从0号索引存储数据，+1是为了存储\0（可选）
3     int length;
4 };
5
6 typedef struct {
7     char *str; // 动态分配空间
8     int length;
9 };
```

1.2 串的初始化

与普通变量赋值操作不同，串的赋值操作不能直接用=来实现，通过定义初始化函数来实现空间拷贝。

```
int strAssign(StrType *str, const char *ch);
```

2. 串的匹配

字符串模式匹配：在主串中找到与模式串相同的子串，并返回其所在位置。

2.1 暴力匹配 (Brute-Force)

- 暴力匹配也称之为朴素模式匹配，其思想如下：
 - 以字符为单位，从左到右移动模式串，直到匹配成功为止。
 - 从左到右进行匹配，如果模式串中的第一个字符匹配成功，这继续往后进行匹配，如果匹配失败，则模式串从文本串的下一个字符进行匹配，一直重复。
 - 直到匹配成功或者匹配完所有的文本串为止。
- 朴素模式匹配算法的缺点：
 - 当某些子串与模式串部分匹配，一旦出现失配时，主串的扫描指针i经常回溯，导致时间开销增加。
 - 最坏时间复杂度 $O(n * m)$

2.2 KMP算法

由D.E.Knuth, J.H.Morris和V.R.Pratt提出，因此称为 KMP算法，他的思想是，不匹配的字符之前，一定是和模式串一致的，是否可以从这个已知信息来确定模式失配时，下次从模式串的第几个位置开始匹配。

2.2.1 跳过不可能成功的字符比较

- 假设模式串为"abcbabd"，主串为"abcbabxxxx"
 - 从主串s[1]开始匹配时，在p[6]时失配
 - 既然在p[6]处失配，那么说明s[1:5]的信息一定是模式串的p[1:5]，所以按照朴素匹配算法，s[2]、s[3]、开始匹配尝试，是不是可以明确肯定不会成功。
 - 而从s[4]开始，有可能成功

2.2.2 KMP算法的思想

- 当子串和模式串不匹配时，主串指针i不回溯，通过改变模式串指针j的值，来确定子串从失配处和模式串的哪个位置进行比较，因为模式串前面的信息我在前面比较的时候已经知道信息了。
- 如果能够存储子串失配后从模式串的哪个位置上进行比较，就可以实现KMP算法，故引入next数组，专门存放这个值。
- 显然，next数组里的值，只跟模式串有关，因为模式串前面已经成功匹配的字符，就表示子串中已经包含了这些字符。

2.2.3 手动算next数组

- 串的前缀：包含第一个字符，且不包含最后一个字符的子串

- 串的后缀：包含最后一个字符，且不包含第一个字符的子串
- 当第j个字符匹配失败，由前[1, j-1]个字符组成的串记为S，手动计算就是根据这个S来决定的
- $next[j]$ 的值：S最长相等前后缀的长度 + 1，表示对于子串中前j-1个字符而言

2.2.4 next数组值的规律

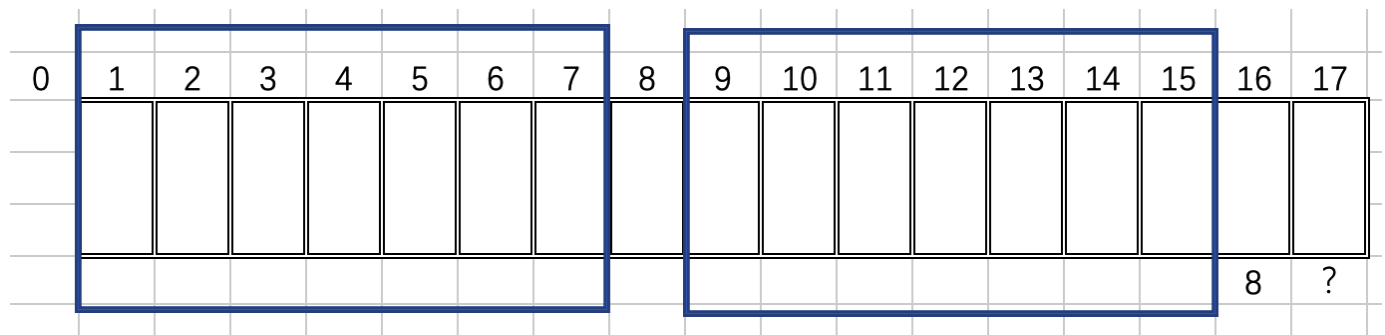
- $next[j]$ 的值每次最多增加1
- 模式串的最后位字符不影响next数组的结果
- next数组的定义：当主串与模式串的某一位字符不匹配时，模式串要回退到的位置

2.2.5 KMP的代码

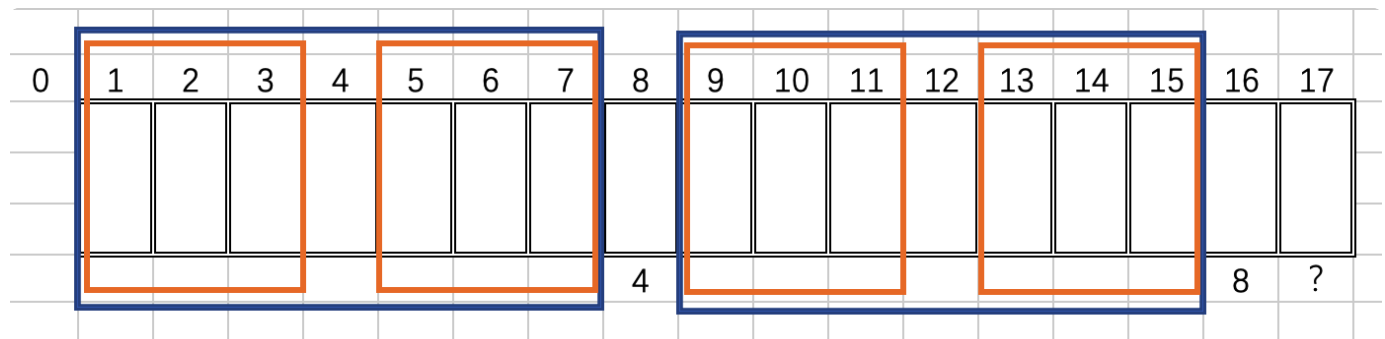
2.3 next数组的代码实现

2.3.1 代码实现逻辑

- 假设模式串要计算 $next[17]$ 的值，那么说明 $next[16]$ 的值是已知的， $next[16]$ 假设等于8，就说明有7个前后缀是相等的



- 如果 $p[8] == p[16]$ ，就说明 $next[17]$ 前面应该有 $8 + 1$ 个匹配的相等前后缀
- 如果 $p[8] != p[16]$ ，那么假设 $p[8] = 4$ ，就说明有3个前后缀是相等的，而 $p[16]$ 又等于8，说明 $p[1:3]$ 和 $p[9:11]$ 应该是相等的，同时 $p[5:7]$ 也应该等于 $p[13:15]$ 。



- 如果 $p[16] == p[4]$ 的话，那么说明 $next[17]$ 前面应该有 $3 + 1$ 个相等前后缀，实际就是 $p[1:4]$ 和 $p[13:16]$ 。

- 如果 $p[4] \neq p[16]$ 的话，那么假设 $p[4] = 2$ ，这样 $p[1]$ 和 $p[3]$ 相等，也等价于 $p[1]$ 和 $p[15]$ 相等
- 如果此时 $p[16] == p[2]$ ，就说明 $p[1:2]$ 和 $p[15:16]$ 相等， $next[17] = 2 + 1$
- 否则还是不等，由于 $next[2] = 1$ 的值，即 $next[1]$ 等于0，那么遇到0，递推就结束了。此时 $next[17] = 1$

2.3.2 求next数组的代码实现

▼ next数组的实现

🔗 复制代码

```
1 void getNext(StrType *subStr, int next[]) {
2     int i = 1, j = 0;    // 串从数组下标1位置开始存储
3     next[1] = 0;
4     while (i < subStr->length) {
5         if (j == 0 || subStr->str[i] == subStr->str[j]) {
6             ++i;
7             ++j;
8             next[i] = j;
9         } else {
10            j = next[j];
11        }
12    }
13 }
```